

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Т.В. Семененко
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Представление данных в ЭВМ типа
VAX- 11. Арифметико-логические
операции

по курсу: Архитектура ЭВМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Перевод исходных данных в 16СС.....	5
3 Ход работы.....	6
4 Вывод.....	10

1 Цель работы

Целью данной лабораторной работы является изучение форматов команд и данных, типов адресации, системы арифметико-логических операций процессора VAX-11. Освоение симулятора процессора VAX-11 и выполнение программы арифметико-логической обработки целочисленных данных с использованием прямой адресации и различных способов косвенной адресации.

2 Задание

Практическая часть работы включает выполнение следующих действий:

1. Формирование числовых значений операндов в соответствии с индивидуальным заданием, перевод их в шестнадцатеричную систему счисления и определение минимального формата представления исходных данных;
2. Определение номеров РОН и адресов в памяти для размещения операндов;
3. Составление и выполнение программы работы с данными, хранящимися в РОН или в памяти с использованием прямой и различных способов косвенной адресации, по заданному алгоритму;
4. Тестирование программы путем ручной трассировки заданного алгоритма с последующим сравнением результатов работы программы с результатами ручной трассировки.

Также предоставлены исходные данные для моего варианта 4321 группы, -
 $NV = 13$, $NGL = 1$, $NGP = 2$

Алгоритм для моего варианта $1 + \text{остаток от } (1+1+2)/9 = 5$ и представлен на рисунке 1. Для полного представления распишем операции:

1. $X1 = -X2$ (отрицание)
2. $X8 = X8 + X7$ (сложение)
3. $X6 = X5 + X4$ (сложение)
4. $X3 = X3 - 1$ (декремент)
5. $X8 = X8 \setminus * 2^{X9'}$ (сдвиг влево, где $X9' = X9 \% 10$)
6. $X7 = X7 + X8 + C$ (сложение с переносом)

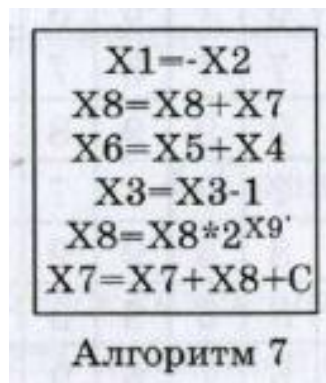


Рисунок 1 – Алгоритм программы

3 Перевод исходных данных в 16СС

Первым этапом является получение десятичных X1-X, исходные данные

$$X1 = (-1)^{13} * [(13 + 2) * 3] = -1 * 45 = -45$$

$$X2 = (-1)^{(13+1)} * [(13 + 1) * 2] = 1 * 28 = 28$$

$$X3 = (-1)^{(13+1)} * [13 + 2 + 1 + 21] = 1 * 37 = 37$$

$$X4 = (-1)^{13} * [13 + 2 + 29]^2 = -1 * 44^2 = -1936$$

$$X5 = (-1)^{(13+1)} * [13 + 1 + 23]^2 = 1 * 37^2 = 1369$$

$$X6 = (-1)^{(13+1)} * [13 + 2 + 1 + 79]^2 = 1 * 95^2 = 9025$$

$$X7 = X4^2 = (-1936)^2 = 3748096$$

$$X8 = -X5^2 = -1369^2 = -1874161$$

$$X9 = (-1)^{(13+1)} * X6^2 = 1 * 9025^2 = 81450625$$

Далее задачей является перевод чисел в шестнадцатеричную СС. Числа в диапазоне [-128...127] будут byte. Числа в диапазоне [-32768...32767] будут word. Числа в диапазоне [-2 147 483 648...2 147 483 647] будут dword. На рисунке 2 показан перевод чисел в 16СС.

Перевод исходных данных в шестнадцатеричную систему счисления выполнен с учетом минимального формата представления каждого операнда: байты для значений в диапазоне от -128 до 127, слова для значений от -32768 до 32767, и длинные слова для больших значений. В таблице представлены все девять операндов с их десятичными значениями, шестнадцатеричными эквивалентами и определенными форматами данных, что необходимо для корректного размещения данных в памяти и регистрах процессора VAX-11.

Переменная	Десятичное значение	Шестнадцатеричное значение	Минимальный формат
X1	-45	D3	Байт
X2	28	1C	Байт
X3	37	25	Байт
X4	-1936	F870	Слово
X5	1369	0559	Слово
X6	9025	2341	Слово
X7	3748096	00393100	Длинное слово
X8	-1874161	FFE3670F	Длинное слово
X9	81450625	04D3A311	Длинное слово (младший байт: 05)

Рисунок 2 – Перевод в 16СС

3 Ход работы

Для начала необходимо определить расположение операнд.

В регистрах (РОН):

- X2: R3 = 1C (байт)
- X5: R2 = 0559 (слово)
- X6: RC (R12) = 2341 (слово)

В памяти:

- X1: адрес 0F, значение D3 (байт)
- X3: адрес 19, значение 25 (байт)
- X4: адрес 23, значение F870 (слово)
- X7: адрес 2D, значение 00393100 (длинное слово)
- X8: адрес 37, значение FFE3670F (длинное слово)
- X9: адрес 41, значение 05 (байт)

Промежуточные ячейки (ПЯ):

Адрес начала ПЯ: $NV * NGR * 1000 = 13 * 2 * 1000 = 26000 = 6590$ (hex),

- ПЯ1 (адрес 402): 0000000F (адрес X1)
- ПЯ2 (адрес 406): 0000002D (адрес X7)
- ПЯ3 (адрес 40C): 00000019 (адрес X3)

Регистры общего назначения:

- R0: 00000025 (адрес X4 для команды 3)
- R1: 0000040C (адрес ПЯ3)
- R2: 00000559 (значение X5)

- R3: 0000001C (значение X2)
- R6: 00000406 (адрес ПЯ2)
- R7: 00000042 (адрес X9)
- R8: 00000402 (адрес ПЯ1)
- R9: 00000037 (адрес X8)
- RC (R12): 00002341 (значение X6)

Способы адресации операндов определяют механизм доступа к данным при выполнении команд программы. В данной работе используются четыре основных способа адресации: прямая регистровая (коды 5 и 6), косвенная с автоувеличением (код 7) и косвенная через память с автоувеличением (код 9). Каждый способ адресации применяется к конкретным операндам согласно варианту комбинации адресации, что позволяет эффективно работать с данными, размещенными как в регистрах, так и в памяти, используя промежуточные ячейки для хранения адресов операндов. Также представим способы адресации операндов на рисунке 3.

Оператор	Способ адресации	Комментарий		
		3 операнд	2 операнд	1 операнд
1	-	5	9	X1 = -X2
2		9	6	X8 = X8 + X7
3	5	5	7	X6 = X5 + X4
4			9	X3 = X3 - 1
5	7	6	6	X8 = X8 * 2 * X9
6		6	9	X7 = X7 + X8 + C

Код	Способ адресации	Мнемоника	Описание
5	Прямая регистровая	R	Операнд в регистре
6	Прямая регистровая	R	Операнд в регистре
7	Косвенная с автоувеличением	(R)+	Адрес в регистре, после выборки регистр увеличивается
9	Косвенная через память с автоувеличением	@(R)+	Адрес ПЯ в регистре, из ПЯ берется адрес операнда, регистр увеличивается

Рисунок 3 – Способы адресации операндов

Карта памяти программы представляет собой полное распределение адресного пространства, включающее размещение всех операндов X1-X9 в памяти и регистрах, адреса промежуточных ячеек для косвенной адресации, а также расположение текста программы в памяти. Карта показывает начальные адреса каждого операнда, их форматы представления (байт, слово, длинное слово), значения в шестнадцатеричной системе счисления, а также назначение

регистров общего назначения для хранения адресов и значений операндов, что обеспечивает полное понимание структуры программы и механизма доступа к данным. На рисунке 4 изображена карта памяти программы.

Объект	Формат	Код	Адрес
X1	B	D3	0F
X2	B	1C	R3
X3	B	25	19
X4	W	F870	23
X5	W	0559	R2
X6	W	2341	RC
X7	L	00393100	2D
X8	L	FFE3670F	37
X9	B	05	41
Адрес X4 (CA = 7)	L	00000025	R0
Адрес X8 (CA = 6)	L	00000037	R9
Адрес X9 (CA = 7)	L	00000041	R7
Текст программы		см. таблицу 2	A8-BA
Адрес ПЯ1	L	00000402	R8
ПЯ1 X1 (CA = 9)	L	0000000F	402
Адрес ПЯ2	L	00000406	R8
ПЯ2 X7 (CA = 9)	L	0000002D	406
Адрес ПЯ3	L	0000040C	R1
ПЯ3 X3 (CA = 9)	L	00000019	40C

Рисунок 4 – Карта памяти

Текст программы представлен в двух формах: мнемонических кодах, которые обеспечивают читаемость и понимание выполняемых операций, и машинных кодах в шестнадцатеричном представлении, которые непосредственно выполняются процессором. Каждая команда содержит код операции, операнды с указанием способов адресации, адрес размещения команды в памяти и комментарий, описывающий выполняемую операцию. Программа состоит из шести команд, реализующих алгоритм обработки данных: MNEGB для отрицания, ADDL2 и ADDW3 для сложения, DECB для декремента, ASHL для арифметического сдвига и ADWC для сложения с переносом. На рисунке 5 изображена результат трассировки программы.

Оператор	Адрес	Шестнадцатеричный код	Мнемонический код	Комментарий
1	A8	8E 53 98	MNEGB R3 @(R8)+	X1 = -X2
2	AB	C0 98 69	ADDL2 @(R8) (R9)+	X8 = X8 + X7
3	AE	A1 70 52 5C	ADDW3 -(R0) R2 RC	X6 = X5 + X4
4	B2	97 91	DECB @(R1)+	X3 = X3 - 1
5	B4	78 77 69 69	ASHL (R9)+ (R9)+ -(R7)	X8 = X8 $\setminus$ * 2 $\setminus$ * X9'
6	B8	D8 96 69	ADWC @(R6)+ (R9)+	X7 = X7 + X8 + C

Рисунок 5 – Текст программы в мнемонических и машинных кодах

Результаты трассировки программы демонстрируют пошаговое изменение значений операндов и регистров при выполнении каждой команды. Таблица трассировки содержит информацию о состоянии операндов до и после выполнения каждой операции, что позволяет отследить все преобразования данных на протяжении выполнения программы. Анализ трассировки подтверждает корректность работы программы, так как все промежуточные и итоговые значения соответствуют ожидаемым результатам вычислений, выполняемых согласно заданному алгоритму. На рисунке 6 результат работы программы в целом.

The screenshot shows a debugger interface with several panels:

- Memory Editor (left):** A table showing memory addresses from 00000000 to 00000130. The value at address 000000B9 is highlighted in red, showing the sequence 52 5C 97 91 78 77 69 69 D8 69 96 00 00 00 00 00.
- Registers and Flags (right):** A list of registers R0 through R7 and their values. R0 is 00000023, R1 is 00000410, R2 is 00000559, R3 is 0000001C, R4 is 00000000, R5 is 00000000, R6 is 0000040A, and R7 is 00000041. Flags T=0, N=0, Z=0, V=0, C=0 are shown.
- Variables (bottom right):** A table listing variables X1_pointer, X1, X1_val, X7, X7_val, X8, X8_val, X7_pointer, X4, and X4_val with their types and values.
- Debug Messages (bottom left):** A section titled 'Отладочные сообщения' showing the execution of a command: 'Описание выполнения команды: Адрес: 000000BB, Код команды: 00, Мнемоника: HALT, Число операндов: 0'.

Рисунок 6 – Результат работы программы

4 Вывод

В ходе выполнения данной работы были получены и закреплены навыки работы с ассемблером для процессоров семейства VAX, а в частности работы с процессором VAX-11. Изучены форматы команд и способы адресации процессора VAX-11: прямая регистровая адресация (способ 6), косвенная адресация с автоувеличением (способ 7), косвенная адресация с автоуменьшением (способ 8) и косвенная адресация через память с автоувеличением (способ 9).

Освоены различные способы косвенной адресации операндов, включая использование промежуточных ячеек (ПЯ) для хранения адресов операндов, автоматическое изменение регистров при использовании способов адресации с автоувеличением/уменьшением, а также косвенная адресация через память для работы с операндами в памяти. Составлена программа для выполнения алгоритма арифметико-логической обработки данных с реализацией операций отрицания (MNEGB), сложения (ADDL2, ADDW3), декремента (DECB), арифметического сдвига (ASHL) и сложения с переносом (ADWC).

Программа протестирована путем ручной трассировки, при этом все результаты выполнения команд совпали с ожидаемыми, а изменения регистров и памяти отслежены на каждом шаге выполнения программы. Изучено представление данных в формате little-endian, где младший байт записывается первым, а длинные слова занимают 4 байта в памяти. Понята структура команд VAX-11, где код операции определяет тип команды, операнды кодируются байтами в зависимости от способа адресации, а размер команды зависит от количества и типов операндов. Работа была выполнена с использованием эмулятора, и запускалась на нем. Были закреплены навыки изучения технической документации, получены навыки работы с различными способами адресации.